

ASSIGNMENT -3.1

ROLL_NO: 2303A51416

NAME: YEROJU ANJALI

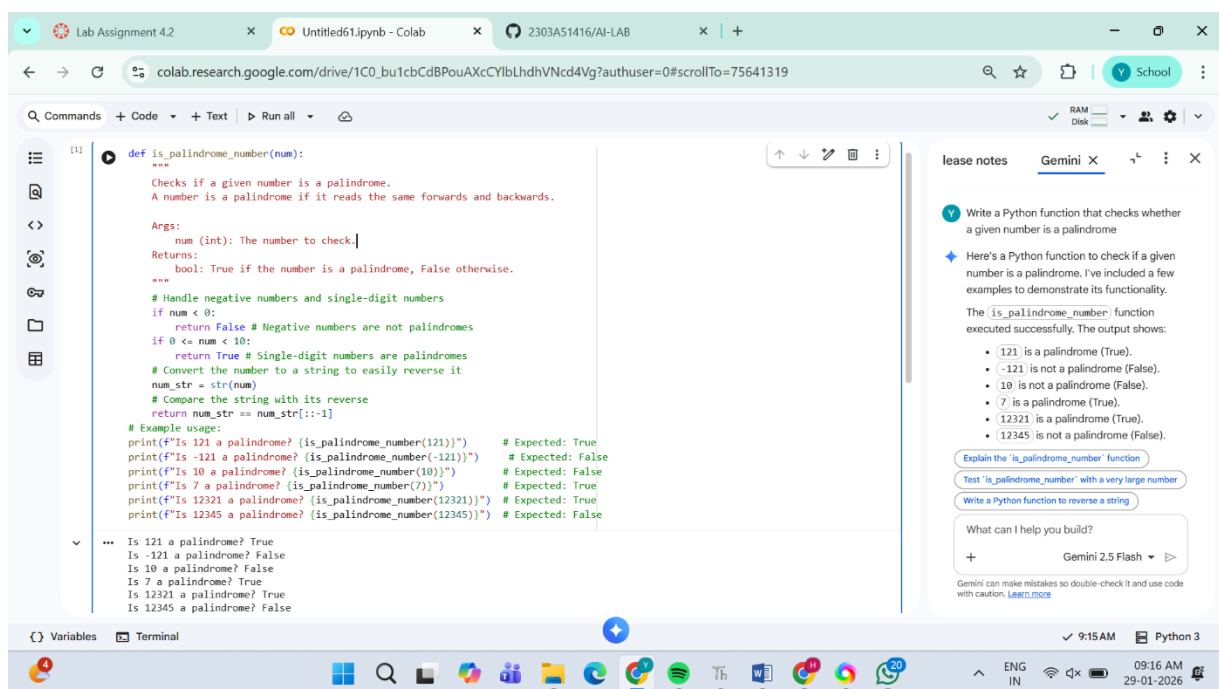
BATCH: 29

TASK-1: ZERO-SHOT PROMPTING (PALINDROME NUMBER PROGRAM)

PROMPT: Write a Python function that checks whether a given number is a palindrome.

The function should return True if it is a palindrome and False otherwise.

CODE:



The screenshot shows a Google Colab notebook with a Python function `is_palindrome_number` and its execution output. The function checks if a number is a palindrome by converting it to a string and comparing it with its reverse. The output shows the function returning True for 121, 10, 7, and 12321, and False for -121, 12345, and 1234.

```
[11] def is_palindrome_number(num):  
    """  
    Checks if a given number is a palindrome.  
    A number is a palindrome if it reads the same forwards and backwards.  
  
    Args:  
        num (int): The number to check.  
    Returns:  
        bool: True if the number is a palindrome, False otherwise.  
    """  
    # Handle negative numbers and single-digit numbers  
    if num < 0:  
        return False # Negative numbers are not palindromes  
    if 0 <= num < 10:  
        return True # Single-digit numbers are palindromes  
    # Convert the number to a string to easily reverse it  
    num_str = str(num)  
    # Compare the string with its reverse  
    return num_str == num_str[::-1]  
  
    # Example usage:  
    print(f"Is 121 a palindrome? {is_palindrome_number(121)}") # Expected: True  
    print(f"Is -121 a palindrome? {is_palindrome_number(-121)}") # Expected: False  
    print(f"Is 10 a palindrome? {is_palindrome_number(10)}") # Expected: False  
    print(f"Is 7 a palindrome? {is_palindrome_number(7)}") # Expected: True  
    print(f"Is 12321 a palindrome? {is_palindrome_number(12321)}") # Expected: True  
    print(f"Is 12345 a palindrome? {is_palindrome_number(12345)}") # Expected: False
```

Output:

```
... Is 121 a palindrome? True  
Is -121 a palindrome? False  
Is 10 a palindrome? False  
Is 7 a palindrome? True  
Is 12321 a palindrome? True  
Is 12345 a palindrome? False
```

OBSERVATION:

-The model is given only the explanation of the question

-Any example or detailed explanation is not given

-Answer is accurate but not specific with negative and nonintegers values

TASK-2: ONE-SHOT PROMPTING (FACTORIAL CALCULATION)

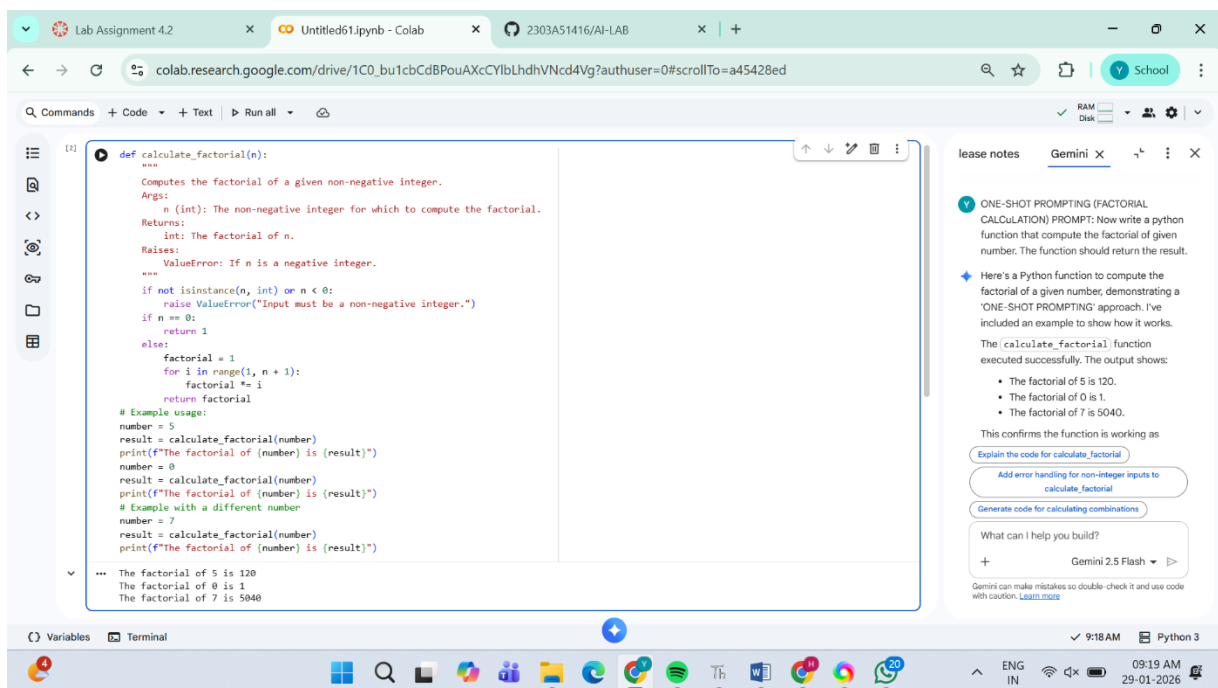
PROMPT: Now write a python function that compute the factorial of given number. The function should return the result.

Example:

Input:5

Output:120

CODE:



The screenshot shows a Google Colab notebook with a Python function `calculate_factorial(n)` defined. The function includes docstrings, type hints, and error handling for non-integer and negative inputs. It uses a loop to calculate the factorial. The output shows the factorial of 5 is 120, 0 is 1, and 7 is 5040. A Gemini chat window on the right provides a one-shot prompt and the resulting code.

```
[2] def calculate_factorial(n):  
    """  
    Computes the factorial of a given non-negative integer.  
    Args:  
        n (int): The non-negative integer for which to compute the factorial.  
    Returns:  
        int: The factorial of n.  
    Raises:  
        ValueError: If n is a negative integer.  
    """  
    if not isinstance(n, int) or n < 0:  
        raise ValueError("Input must be a non-negative integer.")  
    if n == 0:  
        return 1  
    else:  
        factorial = 1  
        for i in range(1, n + 1):  
            factorial *= i  
        return factorial  
  
# Example usage:  
number = 5  
result = calculate_factorial(number)  
print(f"The factorial of {number} is {result}")  
number = 0  
result = calculate_factorial(number)  
print(f"The factorial of {number} is {result}")  
# Example with a different number  
number = 7  
result = calculate_factorial(number)  
print(f"The factorial of {number} is {result}")  
  
... The factorial of 5 is 120  
The factorial of 0 is 1  
The factorial of 7 is 5040
```

ONE-SHOT PROMPTING (FACTORIAL CALCULATION) PROMPT: Now write a python function that compute the factorial of given number. The function should return the result.

Here's a Python function to compute the factorial of a given number, demonstrating a 'ONE-SHOT PROMPTING' approach. I've included an example to show how it works.

The `calculate_factorial` function executed successfully. The output shows:

- The factorial of 5 is 120.
- The factorial of 0 is 1.
- The factorial of 7 is 5040.

This confirms the function is working as

[Explain the code for calculate_factorial](#)

[Add error handling for non-integer inputs to calculate_factorial](#)

[Generate code for calculating combinations](#)

What can I help you build?

+ Gemini 2.5 Flash

Gemini can make mistakes so double-check it and use code with caution. [Learn more](#)

OBSERVATION:

Clear understanding of the output

Better choice of logic-stack overflow, recursion complexity

Correct handling of base case

Improve code simplicity

TASk-3: FEw-SHOT PROMPTING (ARMSTRONG NuMBER CHECK)

Prompt: Example 1:

Input: 153

Output: Armstrong Number

Example 2:

Input: 370

Output: Armstrong Number

Example 3:

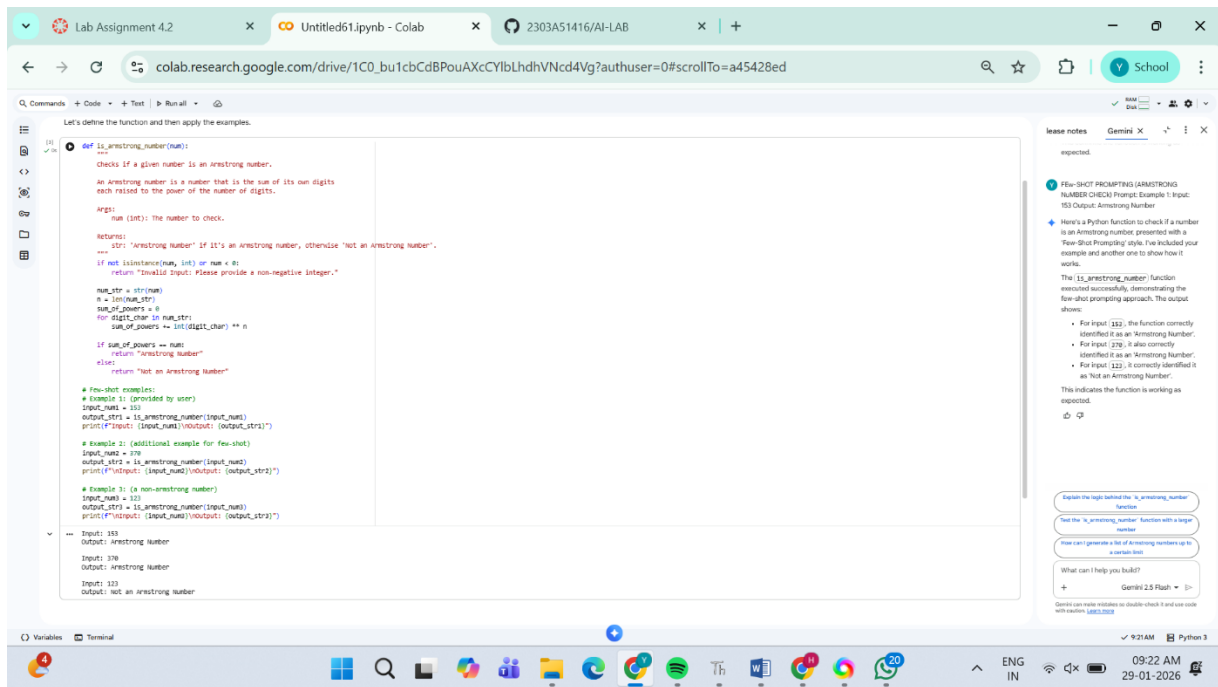
Input: 123

Output: Not an Armstrong Number

Now write a Python function that checks whether a given number is an Armstrong number.

The function should return an appropriate result.

CODE:



OBSERVATION:

Clear output formatting. structured way

Correct logic selection

Easy understanding of code

Exact Appropriate answer

Optimized and customized solution

TASK-4: CONTEXT-MANAGED PROMPTING (OPTIMIZED NUMBER CLASSIFICATION)

PROMPT:

You are writing a Python program for number classification.

Requirements:

- Accept only integer input

- Handle invalid and negative inputs properly
- Classify the number as Prime, Composite, or Neither
- Optimize the logic for efficiency (avoid unnecessary checks)
- Return clear and user-friendly messages
- Write clean and readable Python code

Generate the program accordingly.

CODE:

```
def classify_number(num):
    """Classify a given integer as 'Prime', 'Composite', or 'Neither'."""
    # Returns:
    #   str: The classification of the number (Prime, Composite, or Neither).
    #   Classification rules: 'Neither' for numbers less than or equal to 1
    #   (if num <= 1):
    #       return "Neither"
    #   Prime numbers: 2 and 3
    #   (if num == 2 or num == 3):
    #       return "Prime"
    #   Composite numbers: multiples of 2 (except 2) and multiples of 3 (except 3)
    #   (if num % 2 == 0 or num % 3 == 0):
    #       return "Composite"
    #   Optimized check for prime numbers using 6k +/- 1 rule
    #   Check divisibility by numbers of the form 6k +/- 1 up to sqrt(num)
    i = 5
    while i * i <= num:
        # Stop checking as soon as a divisor is found
        if num % i == 0 or num % (i + 2) == 0:
            return "Composite"
        i += 6
    return "Prime"

def main():
    """Main function to get user input, classify the number, and display the result.
    Note: User should input a positive integer.
    """
    print("Number Classification Program")
    print("Enter an integer to classify (or 'q' to quit).")

    while True:
        user_input = input("Enter an integer: ")
        if user_input.lower() == 'q':
            print("Exiting program. Goodbye!")
            break

        try:
            number = int(user_input)
            classification = classify_number(number)
            print(f"The number {number} is classified as: {classification}")
        except ValueError:
            print("Invalid input. Please enter a valid integer or 'q' to quit.")

    if __name__ == "__main__":
        main()

# Number Classification Program
# Enter an integer to classify (or 'q' to quit).
# Enter an integer: 25
# The number 25 is classified as: Composite
# Enter an integer: 29
# The number 29 is classified as: Prime
```

OBSERVATION:

The role is defined

Constraints are clearly stated

Efficiency and validation of the code

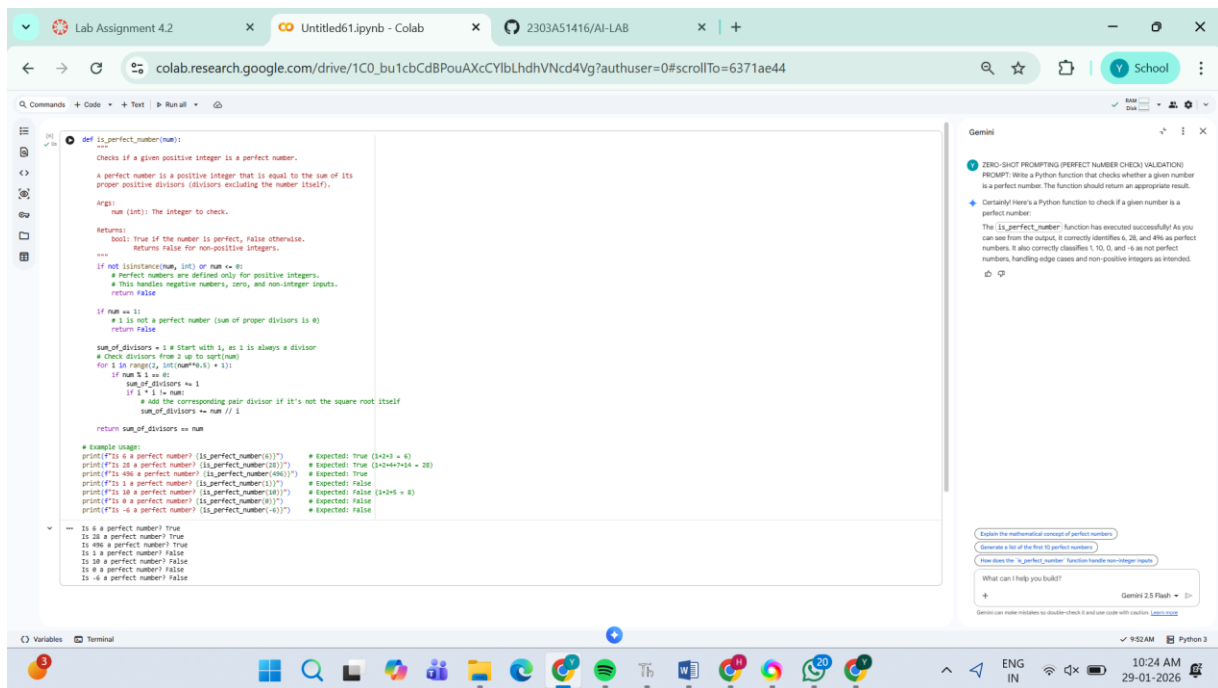
But the inputs should be specified more clearly mentioned

TASK-5: ZERO-SHOT PROMPTING (PERFECT NUMBER CHECK) VALIDATION)

PROMPT: Write a Python function that checks whether a given number is a perfect number.

The function should return an appropriate result.

CODE:



The screenshot shows a Google Colab notebook titled 'Untitled61.ipynb'. The code defines a function `is_perfect_number(num)` that checks if a given positive integer is a perfect number. The function includes comments explaining its logic and returns `True` if the number is perfect and `False` otherwise. The code also includes example usage and expected output for various inputs.

```
def is_perfect_number(num):  
    """  
    Checks if a given positive integer is a perfect number.  
    A perfect number is a positive integer that is equal to the sum of its  
    proper positive divisors (divisors excluding the number itself).  
    Args:  
        num (int): The integer to check.  
    Returns:  
        bool: True if the number is perfect, False otherwise.  
    """  
    if not isinstance(num, int) or num <= 0:  
        # Perfect numbers are defined only for positive integers.  
        # This handles negative numbers, zero, and non-integer inputs.  
        return False  
    if num == 1:  
        # 1 is not a perfect number (sum of proper divisors is 0)  
        return False  
    sum_of_divisors = 1 # Start with 1, as 1 is always a divisor  
    # Check divisors from 2 up to sqrt(num)  
    for i in range(2, int(num**0.5) + 1):  
        if num % i == 0:  
            sum_of_divisors += i  
            # Add the corresponding pair divisor if it's not the square root itself  
            sum_of_divisors += num // i  
    return sum_of_divisors == num  
  
# Example usage:  
print("Is 6 a perfect number? (is_perfect_number(6))") # Expected: True (1+2+3 = 6)  
print("Is 28 a perfect number? (is_perfect_number(28))") # Expected: True (1+2+4+7+14 = 28)  
print("Is 496 a perfect number? (is_perfect_number(496))") # Expected: True  
print("Is 1 a perfect number? (is_perfect_number(1))") # Expected: False  
print("Is 10 a perfect number? (is_perfect_number(10))") # Expected: False (1+2+5 = 8)  
print("Is 0 a perfect number? (is_perfect_number(0))") # Expected: False  
print("Is -4 a perfect number? (is_perfect_number(-4))") # Expected: False  
  
# Test cases:  
print("Is 6 a perfect number? True")  
print("Is 28 a perfect number? True")  
print("Is 496 a perfect number? True")  
print("Is 1 a perfect number? False")  
print("Is 10 a perfect number? False")  
print("Is 0 a perfect number? False")  
print("Is -4 a perfect number? False")
```

The output of the code is shown in the terminal at the bottom of the notebook, displaying the results of the function calls for various inputs.

OBSERVATION

No input validation – if negative or float any..

Inefficient for large input

Did not specify input constraints

No edge case handling seen

TASK-6: FEW-SHOT PROMPTING (EVEN OR ODD CLASSIFICATION WITH VALIDATION)

PROMPT:

Example 1:

Input: 8

Output: Even

Example 2:

Input: 15

Output: Odd

Example 3:

Input: 0

Output: Even

Now write a Python program that determines whether a given number is Even or Odd.

The program should include proper input validation and return clear messages.

CODE:

The screenshot displays a Google Colab notebook environment. The main code area contains a Python function `classify_even_odd` that takes an integer `number` as input and returns a string indicating if it's 'Even' or 'Odd'. The function includes error handling for non-integer inputs. Below the function definition, there are example usage snippets and additional examples demonstrating the function's behavior with various inputs, including negative integers and non-integer values like 10.5. The output of the code execution is visible at the bottom of the code cell, showing the function's output for each example input.

```
def classify_even_odd(number):  
    """  
    Classifies a given integer as 'Even' or 'Odd'.  
    """  
    Args:  
        number (int): The integer to classify.  
    Returns:  
        str: 'Even' if the number is even, 'Odd' if the number is odd.  
        Handles non-integer input gracefully.  
    """  
    if not isinstance(number, int):  
        return "Invalid input: Please provide an integer."  
    if number % 2 == 0:  
        return "Even"  
    else:  
        return "Odd"  
  
# Example usage:  
input_num = 8  
output_classification = classify_even_odd(input_num)  
print(f"Input: {input_num}")  
print(f"Output: {output_classification}")  
  
# Additional examples:  
print(f"Input: 7, Output: {classify_even_odd(7)}")  
print(f"Input: 0, Output: {classify_even_odd(0)}")  
print(f"Input: -3, Output: {classify_even_odd(-3)}")  
print(f"Input: 10.5, Output: {classify_even_odd(10.5)}")
```

The output of the code execution is as follows:

```
Input: 8  
Output: Even  
Input: 7, Output: Odd  
Input: 0, Output: Even  
Input: -3, Output: Odd  
Input: 10.5, Output: Invalid input: Please provide an integer.
```

On the right side of the notebook, the Gemini AI chat interface is visible. It shows a conversation where the user asks for a Python function to classify even or odd numbers, and the Gemini AI responds with the provided function and an example based on the input.

OBSERVATION:

Negative integer are handled correctly

Program safely rejected non integer inputs

Improve input handling

Clear and consistent output